



Introdução à Programação C

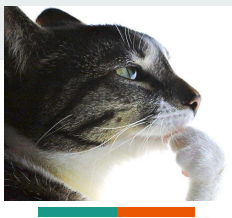
Aula 1



Introdução à Programação C

Aula 1

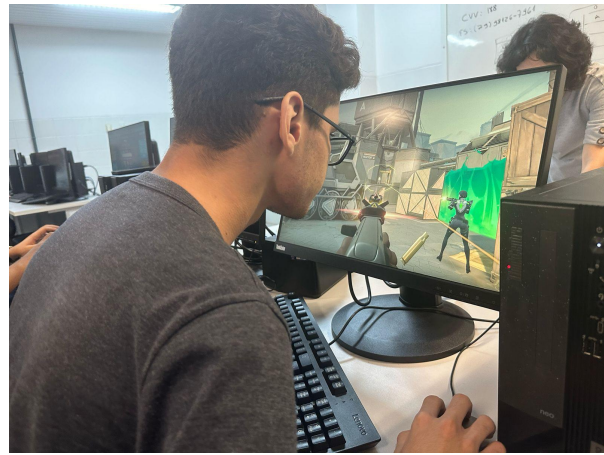
***MUITO OBRIGADO LUCAS
CALUMBI PELOS SLIDES**



Quem sou eu?

- Estudante de **Ciência da Computação** (6º Período)
- **Monitor** de Programação C
- Membro Efetivo na **Liga Acadêmica de Desenvolvimento Web (LAWD)**
- Adoro resolver desafios de **programação competitiva** 🎈

Esse sou eu





Tópicos a serem mencionados:

- Estrutura Básica de um Programa em C
- Terminal Linux
- Compilação e Execução de um Programa em C
- Tipos de variáveis
- IO (entrada e saída)
- Operadores básicos em C



Programa em C

Entendendo a estrutura
básica de um programa em
C

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

```
C programa.c X
C programa.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Cabeçalho do programa

É aqui que incluimos bibliotecas

É na biblioteca “stdio.h”, por exemplo, que está a função “printf”

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```



Assinatura da função "main"

A função "main" é a função principal, é por ela que seu programa começa a ser executado

A função "main" retorna um inteiro (int)

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Escopo da função "main"

Delimita o trecho de código
que pertence a função "main"

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Corpo da função "main"

É onde escrevemos o que
queremos que nosso programa
faça

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Retorno da função "main"

Indica o final do programa

Retorna o valor zero para o SO
indicando que programa
encerrou sem erros

Como se chama cada parte da estrutura de um programa em C?

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

1. ...

2. ...

3. ...

4. ...

5. ...

Como se chama cada parte da estrutura de um programa em C?

```
C programa.c X
C programa.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

1. Cabeçalho
2. Assinatura da função main
3. Escopo da função main
4. Corpo da função main
5. Retorno da função main

Terminal Linux

Comandos de navegação





lucasalumbi@pop-os: ~/Desktop/mini-curso-C



```
lucasalumbi@pop-os:~$ pwd #printa o caminho do diretório (pasta) atual
/home/lucasalumbi
```

```
lucasalumbi@pop-os:~$ ls #lista as pastas e arquivos do diretório atual
```

Arduino	Downloads	Projects	Templates
College	Music	Public	Unity
Desktop	Pictures	snap	Videos
Distros	Programacao-Competitiva	Studies	'VirtualBox VMs'
Documents	Programs	Temp	vmware

```
lucasalumbi@pop-os:~$ cd Desktop #troca para o diretório especificado
```

```
lucasalumbi@pop-os:~/Desktop$ cd .. #troca para o diretório pai
```

```
lucasalumbi@pop-os:~$ cd Desktop/
```

```
lucasalumbi@pop-os:~/Desktop$ ls
```

```
mini-curso-C
```

```
lucasalumbi@pop-os:~/Desktop$ cd mini-curso-C/
```

```
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ls
```

```
programa.c
```

```
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ code . #abre o vscode na pasta atual
```

```
lucasalumbi@pop-os:~/Desktop/mini-curso-C$
```



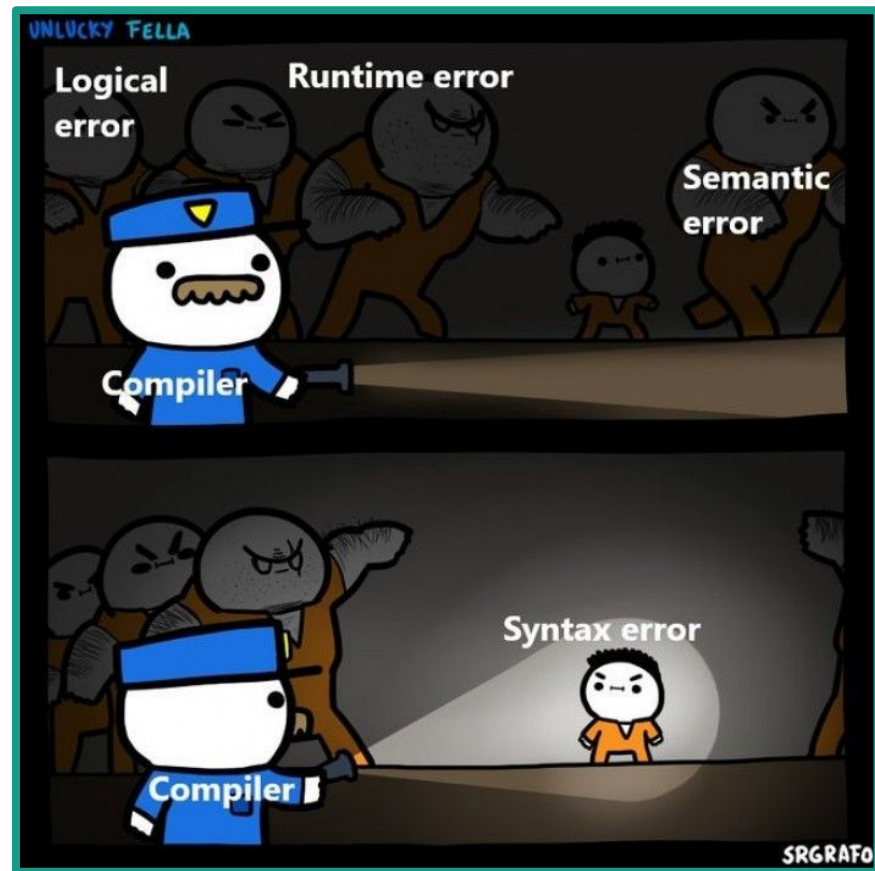
mini-curso-C



Prática: Feche o VScode e o abra através do terminal

Compilação e Execução

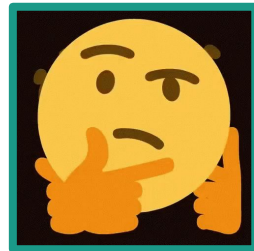
Compiladores...
O que são?
Onde vivem?
O que comem?



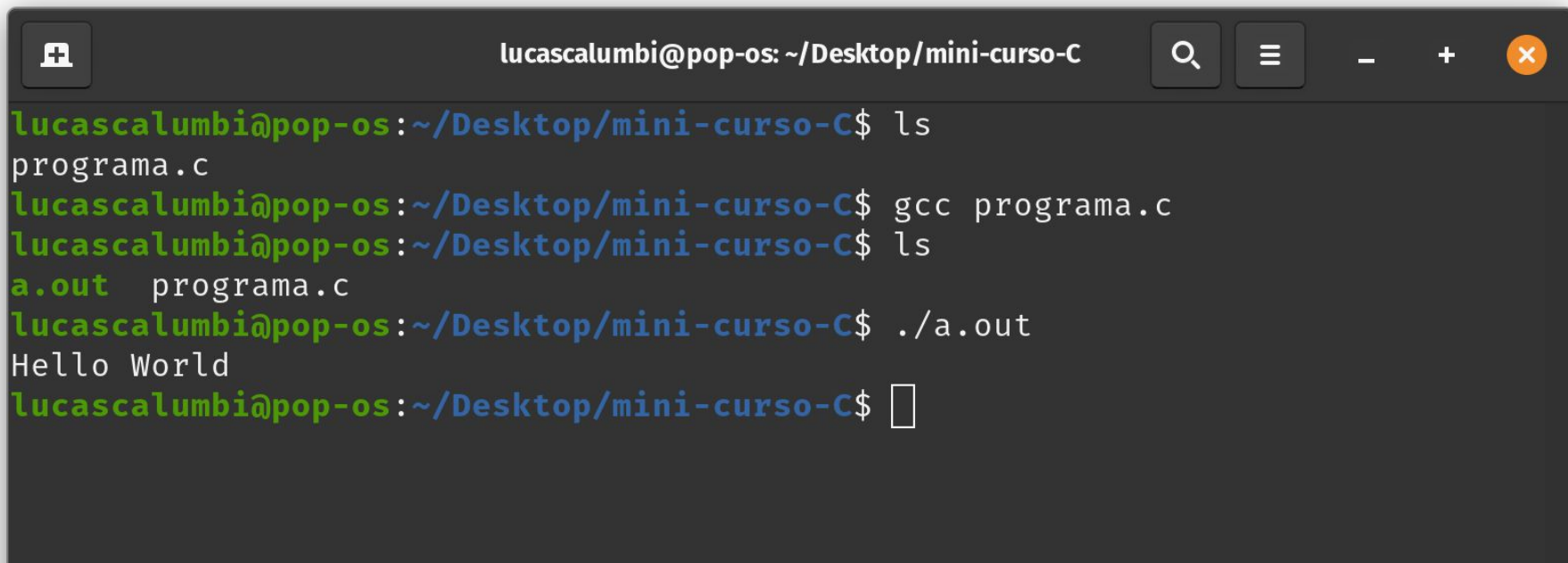
O que é um compilador?

- O compilador é um programa, que pega um arquivo de texto e o transforma em um programa executável

Se o compilador é um programa... e a gente precisa de um compilador para criar programas... como surgiu o primeiro compilador?

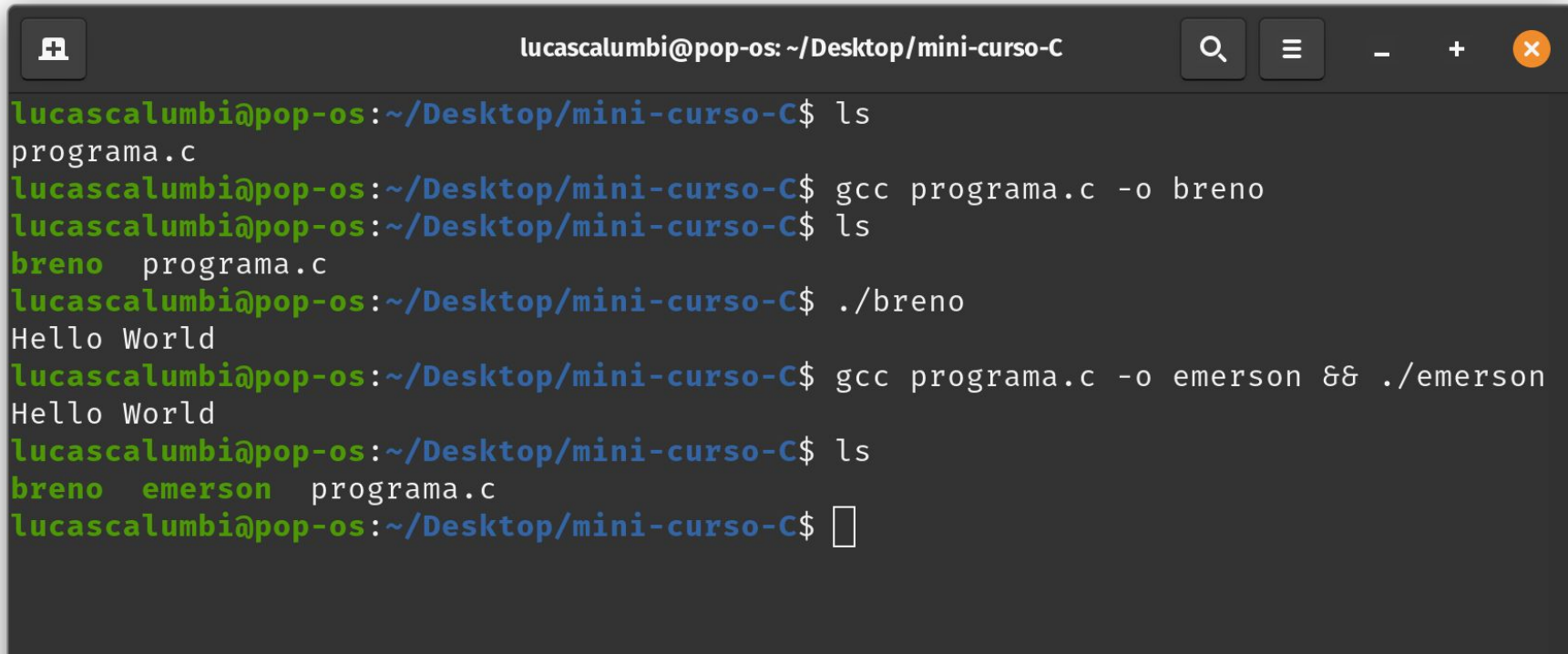


Como compilar e executar um programa em C

A terminal window with a dark background and light text. The title bar at the top reads 'lucasalumbi@pop-os: ~/Desktop/mini-curso-C'. On the left of the title bar is a window control icon (a square with a plus sign). On the right are search, menu, and window management icons (magnifying glass, three horizontal lines, minus, plus, and a red circle with a white 'x'). The terminal content shows a series of commands and their outputs. The prompt is 'lucasalumbi@pop-os:~/Desktop/mini-curso-C\$'. The first command is 'ls', which outputs 'programa.c'. The second command is 'gcc programa.c'. The third command is 'ls', which outputs 'a.out' and 'programa.c'. The fourth command is './a.out', which outputs 'Hello World'. The prompt is then followed by a cursor icon (a white square).

```
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ls
programa.c
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ gcc programa.c
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ls
a.out  programa.c
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ./a.out
Hello World
lucasalumbi@pop-os:~/Desktop/mini-curso-C$
```

E se eu quiser nomear o programa executável?



```
lucasalumbi@pop-os: ~/Desktop/mini-curso-C
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ls
programa.c
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ gcc programa.c -o breno
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ls
breno  programa.c
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ./breno
Hello World
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ gcc programa.c -o emerson && ./emerson
Hello World
lucasalumbi@pop-os:~/Desktop/mini-curso-C$ ls
breno  emerson  programa.c
lucasalumbi@pop-os:~/Desktop/mini-curso-C$
```



**Prática: Mude a mensagem a ser
impressa pelo seu programa, compile-o
e execute-o**



Variáveis e IO

Agora a brincadeira começa...

Mostrando na prática, vamos programar!!!

C programa.c X

C programa.c > main()

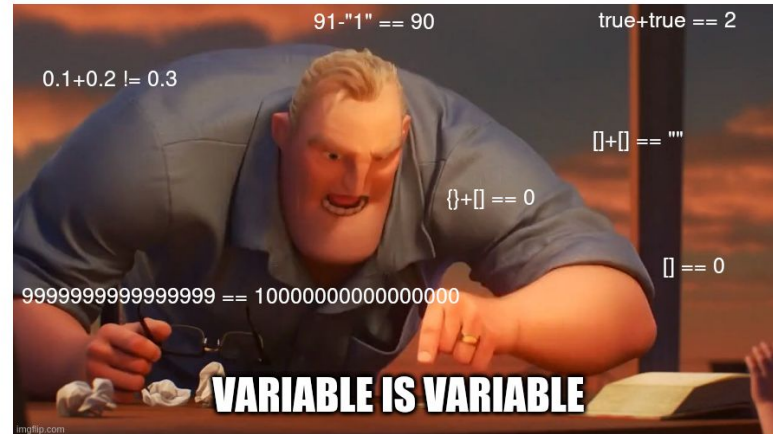
```
1  #include <stdio.h>
2
3  int main()
4  {
5      int num = 0;
6
7      printf("Digite um numero:\n");
8
9      scanf("%d",&num);
10
11     printf("O numero digitado eh %d\n",num);
12
13     return 0;
14 }
15
16
```

Variáveis

O que são variáveis?

C Programmers: All variables must be strictly typed

Javascript programmers:



O que são variáveis?

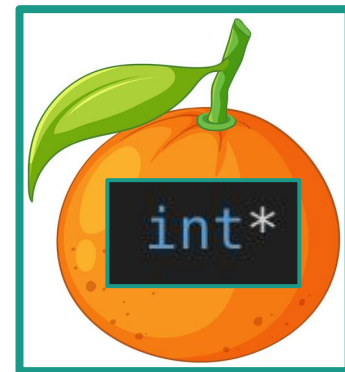
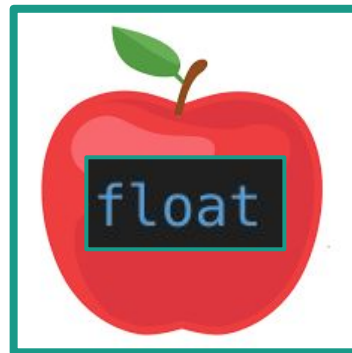
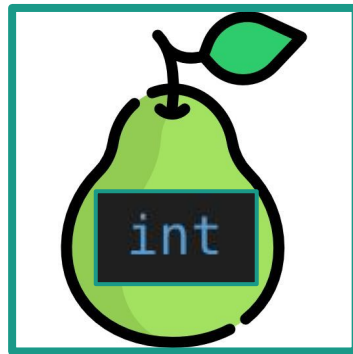
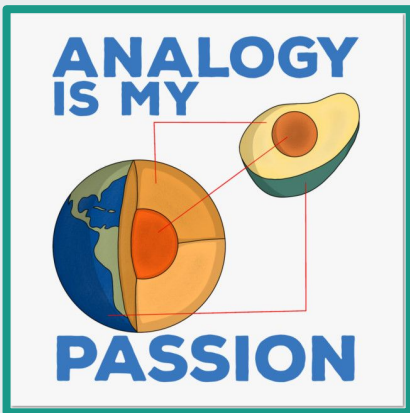


De forma simplificada, uma variável é formada por quatro partes:

1. Tipo
2. Nome
3. Conteúdo (valor)
4. Endereço

Tipos de Variáveis

Tipos são como frutas...



Tipos Básicos de Variáveis

Tipo	Máscara de formatação	Tamanho (Bytes)	Descrição	Exemplo
char	%c	1	caractere	'L'
int	%d	4	inteiro	2025
float	%f	4	real	3.14
double	%lf	8	real	2.71828
char[]	%s	depende	cadeia de caracteres	"muito facil"
(tipo)*	%p	8	endereço de memória	0x7ffcb98e14a4

Intervalos das Variáveis Numéricas

Tipo	Intervalos
char	de -128 a 127 (ASCII)
int	de -2147483648 a 2147483647
long long	de -9223372036854775808 a 9223372036854775807
float	de 1.2e-38 até 3.4e+38
double	de 1.8e-308 a 1.8e+308

Declaração e Inicialização

```
char c = 'L';  
int num = 2025;  
float pi = 3.14f;  
double euler = 2.71828;  
char frase[] = "absurdamente facil";  
int* pnum = &num;
```



Como funciona a memória do computador?

Memória Ram (fictícia)

H000	####
...	####
S008	####
S004	####
S000	####



Vamos analisar
esse código para
entender



```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```



```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```

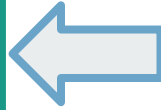
Memória Ram (fictícia)

H000	####
...	####
S008	####
S004	####
S000	####



```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```



Memória Ram (fictícia)

H000	####
...	####
S008	####
S004	####
S000	10



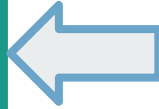
```
&a = S000;  
a = 10;
```





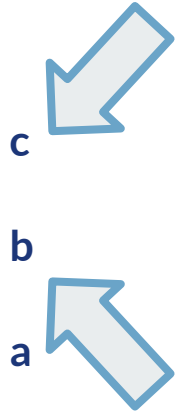
```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```



Memória Ram (fictícia)

H000	####
...	####
S008	####
S004	####
S000	10

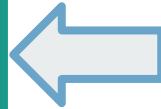


```
&a = S000; a = 10;  
&b = S004; b = ####;  
&c = S008; c = ####;
```



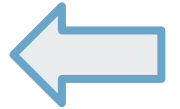
```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```



Memória Ram (fictícia)

H000	####	
...	####	
S008	####	c
S004	20	b
S000	10	a



```
&a = S000; a = 10;  
&b = S004; b = 20;  
&c = S008; c = ####;
```

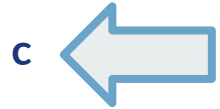


```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```

Memória Ram (fictícia)

H000	####
...	####
S008	30
S004	20
S000	10



b

a

```
&a = S000; a = 10;  
&b = S004; b = 20;  
&c = S008; c = 30
```



```
#include <stdio.h>
```

```
int main(void)
{
```

```
    int a = 10;
    int b, c;
```

```
    printf("&a = %p; a = %d\n", &a,a);
    printf("&b = %p; b = %d\n", &b,b);
    printf("&c = %p; c = %d\n\n", &c,c);
```

```
    b = 20;
    c = a + b;
```

```
    printf("&a = %p; a = %d\n", &a,a);
    printf("&b = %p; b = %d\n", &b,b);
    printf("&c = %p; c = %d\n\n", &c,c);
```

```
    return 0;
```

```
}
```

Exemplo real:

Execução no terminal

```
&a = 0x7ffdc1e00a2c; a = 10
&b = 0x7ffdc1e00a28; b = -1042281640
&c = 0x7ffdc1e00a24; c = 0
```

```
&a = 0x7ffdc1e00a2c; a = 10
&b = 0x7ffdc1e00a28; b = 20
&c = 0x7ffdc1e00a24; c = 30
```

Agora vocês sabem o que são variáveis



Uma variável é uma região da memória (**endereço**), identificada por um **nome**, interpretada de acordo com um **tipo**, e que armazena um valor (**conteúdo**).

Tipo booleano

Em C não temos o tipo *bool*, ao invés disso usamos o tipo *int* da seguinte forma:

- Se for zero, é falso
- Se não for zero, é verdade

Expressões lógicas verdadeiras tem valor 1, enquanto que expressões lógicas falsas tem valor 0

```
int num = 0;
scanf("%d",&num);

int eh_par = num % 2 == 0;

printf("%d\n",eh_par);
```

```
lucasalumbi@pop-os:
5
0
lucasalumbi@pop-os:
6
1
```

Tipo booleano

exemplo de uso

```
int idade = 0;
scanf("%d",&idade);

int eh_adulto = idade >= 18;

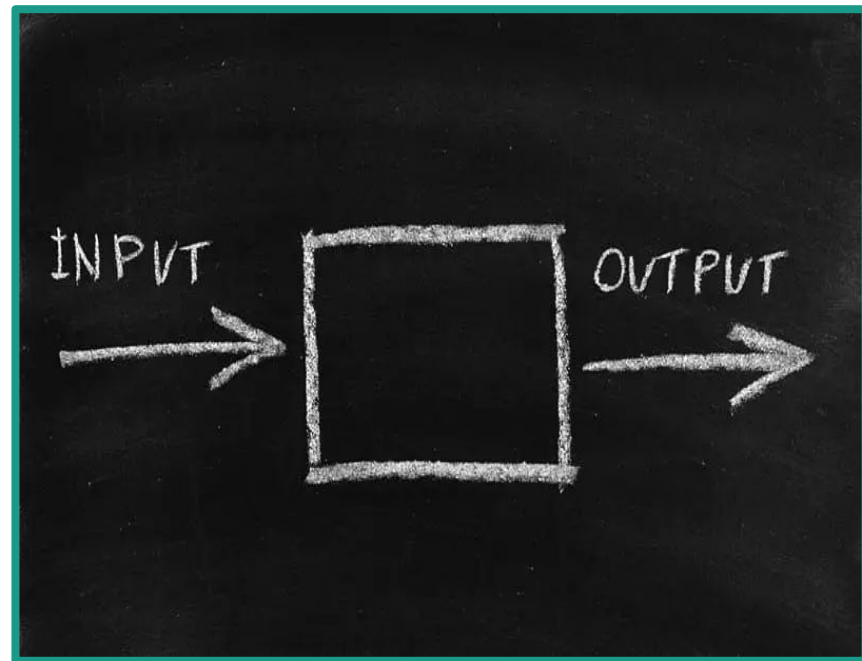
if(eh_adulto)
    printf("eh adulto\n");
else
    printf("nao eh adulto\n");
```



```
lucasalumbi@pop-os:
12
nao eh adulto
lucasalumbi@pop-os:
21
eh adulto
lucasalumbi@pop-os:
```

Entrada e Saída (IO)

printf e scanf



printf

Exibe uma mensagem FORMATADA no terminal

scanf

Escaneia uma mensagem FORMATADA do terminal

printf

sintaxe geral

```
printf("formato", argumentos);
```

- "formato" é uma string que pode conter máscaras
- "argumentos" são as variáveis a serem exibidas pelas máscaras

exemplos de uso

```
int a = 10, b = 20;
```

```
printf("a = %d, b = %d, a+b = %d", a, b, a+b);
```

Prática:

Declare e inicialize uma variável de cada tipo e, usando printf, exiba o conteúdo de cada variável com a sua respectiva máscara de formatação

Tipo	char	int	float	double	char[]	(tipo)*
Máscara	%c	%d	%f	%lf	%s	%p

scanf

sintaxe geral

```
scanf("formato", &argumentos);
```

- "formato" é uma string que pode conter máscaras
- "&argumentos" são os **endereços** das variáveis que terão os valores alterados

```
int a, b;  
scanf("a = %d, b = %d", &a, &b);  
  
printf("a+b = %d\n", a+b);
```

```
lucasalumbi@pc  
a = 2, b = 5  
a+b = 7  
lucasalumbi@pc
```

Prática:

Leia uma frase formatada da seguinte forma:

"Meu nome eh [nome], eu tenho [idade] anos e [altura]m de altura"

E então exiba o nome, idade e a altura.

Tipo	char	int	float	double	char[]	(tipo)*
Máscara	%c	%d	%f	%lf	%s	%p

Entrada	Saída
Meu nome eh Lucas, eu tenho 21 anos e 1.90m de altura	Lucas 21 1.90

Operadores básicos

Operators in C

Unary Operator

++ , -

Unary Operator

Binary Operator

+, -, *, /, %

Arithmetic Operator

<, <=, >, >=, ==, !=

Rational Operator

&&, ||, !

Logical Operator

&, |, <<, >>, ~, ^

Bitwise Operator

=, +=, -=, *, /=, %=

Assignment Operator

Ternary Operator

Size of, comma (,), conditional(? :),
dot (.), arrow (->), cast (type),
addressof (&), dereference (*)

Other Operator

Operadores básicos

C conta com uma larga gama de operadores.

Nessa aula vamos ver os operadores aritméticos, unários e de atribuição.

Tipo	Aritmético	Aritmético	Aritmético	Aritmético	Aritmético	Unário	Unário	Atribuição
Operador	+	-	*	/	%	++	--	=

Exemplo



```
1  int a = 1;  
2  int b,c,d,e,f;  
3  b = a + 1;  
4  c = a - 1;  
5  d = a * b;  
6  e = b / d;  
7  f = a % b;  
8  a++;  
9  a--;
```

```
• > gcc teste.c  
• > ./a.out  
a = 1  
b = 2  
c = 0  
d = 2  
e = 1  
f = 1
```




Também temos
variações do
operador de
atribuição:



```
1  int a = 68;  
2  a += 6;  
3  a -= 7;  
4  printf("a = %d\n", a);
```

```
• > gcc teste.c
```

```
• > ./a.out
```

```
a = 67
```

Atividades Práticas:

- Faça um programa que leia um inteiro A, e exiba A^2 e A^3
- Faça um programa que leia um inteiro A e exiba o último dígito de A
- Faça um programa que leia dois inteiros A e B, e exiba a divisão e o resto de A por B
- Faça um programa que leia o raio de um círculo e calcule a área e o perímetro
- Faça um programa que leia três notas, calcule e exiba a média
- Faça um programa que leia o peso (em kg) e a altura (em metros) e exiba o imc correspondente

Atividade Para Casa:

No período de 2026.2 o prof. Calebe decidiu avaliar seus alunos de uma forma diferente na disciplina de Arquitetura de Computadores.

A nova forma de avaliação consistia em uma única unidade com várias etapas, tendo ao total 4 trabalhos (T1, T2, T3, T4), 1 seminário (S) e 2 provas (P1 e P2), todas podendo ser avaliadas de 0 a 10.

A nota final da disciplina era calculada da seguinte forma:

$$NF \text{ (Nota Final)} = (NT*4 + NS*2 + NP*4) / 10$$

Sendo:

$$NT \text{ (Nota dos trabalhos)} = (T1*1 + T2*2 + T3*3 + T4*2) / 8$$

$$NS \text{ (Nota do seminário)} = S$$

$$NP \text{ (Nota das provas)} = (P1*4 + P2*6) / 10$$

Devido a quantidade de alunos e notas, a planilha de Calebe ficou cheia de linhas e colunas tornando difícil a leitura e cálculo das notas finais.

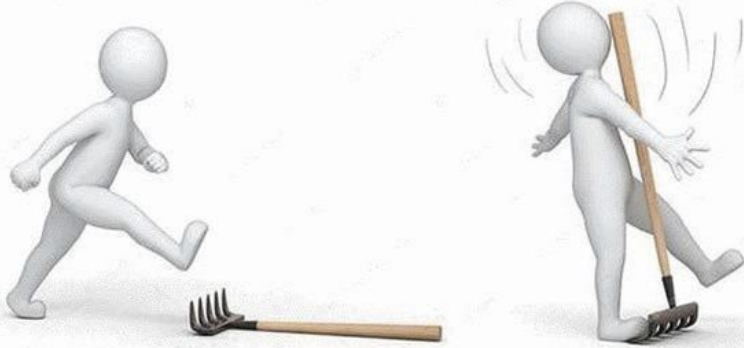
Para ajudar o prof. Calebe nas próximas turmas de Arquitetura, crie um programa que leia as notas de T1, T2, T3, T4, S, P1 e P2 de um aluno e exiba a NT, NS, NP e a NF com precisão de duas casas decimais.

Entrada	Saída
10 10 10 10 0 0 0	NT = 10.00 NS = 0.00 NP = 0.00 NF = 4.00
0 0 0 0 10 0 0	NT = 0.00 NS = 10.00 NP = 0.00 NF = 2.00
0 0 0 0 0 10 10	NT = 0.00 NS = 0.00 NP = 10.00 NF = 4.00

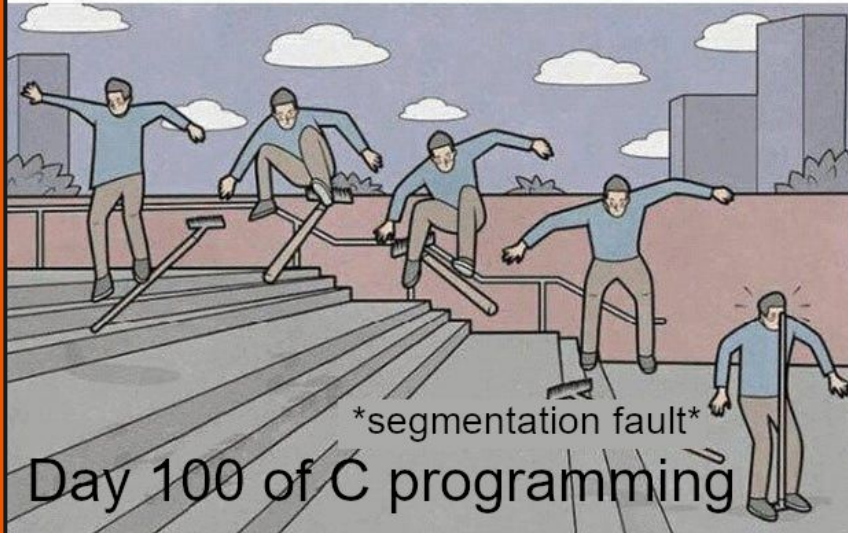
Entrada	Saída
9 8 9 10 8 8 5	NT = 9.00 NS = 8.00 NP = 6.20 NF = 7.68
0 5 6 10 5 8 6	NT = 6.00 NS = 5.00 NP = 6.80 NF = 6.12
7.5 5.4 3.2 2.1 3.7 5.8 4.2	NT = 4.01 NS = 3.70 NP = 4.84 NF = 4.28



segmentation fault



Day 1 of C programming



Day 100 of C programming

Obrigado pela
atenção!